

# Node.js & Express

Construire des APIs REST robustes côté serveur

|         |         |          |            |
|---------|---------|----------|------------|
| Node.js | Express | REST API | Middleware |
|---------|---------|----------|------------|

|     |      |      |
|-----|------|------|
| npm | HTTP | JSON |
|-----|------|------|

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

## ■ Compétences visées

- ✓ Expliquer le modèle d'exécution de Node.js (event loop, non-blocking I/O)
- ✓ Construire une API REST complète avec Express.js
- ✓ Concevoir et implémenter un système de middleware en couches
- ✓ Sécuriser une API avec authentification JWT et validation d'entrées
- ✓ Gérer les erreurs de façon cohérente et standardisée
- ✓ Structurer un projet Express pour la maintenabilité et la scalabilité

# Table des matières

---

## Chapitre 1 — JavaScript côté serveur — le modèle Node.js

- 1.1 L'event loop expliqué
- 1.2 I/O non-bloquante et callbacks
- 1.3 Async/Await — la syntaxe moderne
- 1.4 npm et l'écosystème

## Chapitre 2 — Construire une API REST avec Express

- 2.1 Qu'est-ce que REST ?
- 2.2 Routing et méthodes HTTP
- 2.3 Paramètres, query strings, body
- 2.4 Codes de statut et réponses JSON

## Chapitre 3 — Middleware — le cœur d'Express

- 3.1 La chaîne de middleware
- 3.2 Middleware intégrés et tiers
- 3.3 Écrire ses propres middlewares
- 3.4 Gestion des erreurs

## Chapitre 4 — Authentification et sécurité

- 4.1 JWT — vérification côté serveur
- 4.2 Validation des entrées
- 4.3 Rate limiting et CORS
- 4.4 Variables d'environnement

## Chapitre 5 — Architecture et organisation du code

- 5.1 Structure de dossiers
- 5.2 Séparation routes / services / lib
- 5.3 Connexion à une base de données

## 5.4 Logging et observabilité



# JavaScript côté serveur — le modèle Node.js

## 1.1 L'event loop expliqué

Node.js est un runtime JavaScript construit sur le moteur V8 de Chrome. Sa caractéristique la plus distinctive est son modèle d'exécution single-threaded, non-bloquant, orienté événements. Comprendre l'event loop est indispensable pour écrire du code Node.js performant et éviter les erreurs classiques.

L'event loop est une boucle infinie qui surveille deux structures : la call stack (pile d'appels) et la queue d'événements. Le code JavaScript s'exécute dans la call stack. Quand une opération I/O (lecture de fichier, requête réseau, requête base de données) est initiée, Node.js la délègue au système d'exploitation et continue d'exécuter d'autres tâches. Quand l'opération I/O est terminée, son callback est placé dans la queue d'événements. L'event loop le transfère dans la call stack dès qu'elle est vide.

### Définition — Non-blocking I/O

En Node.js, une opération I/O (réseau, disque, base de données) ne bloque JAMAIS le thread principal. Node.js lance l'opération, enregistre un callback, et continue immédiatement. Quand l'opération se termine, le callback est appelé. Un seul thread peut ainsi gérer des milliers de connexions simultanées sans être bloqué.

*Ordre d'exécution dans l'event loop*

```
// Illustration de l'event loop

console.log('1 - Début');

setTimeout(() => console.log('3 - setTimeout (0ms)'), 0);

fetch('https://api.example.com/data')
  .then(res => res.json())
  .then(data => console.log('4 - Réponse HTTP:', data));

Promise.resolve().then(() => console.log('2.5 - Microtask (Promise)'));

console.log('2 - Fin du code synchrone');

// Ordre d'exécution :
// 1 → 2 → 2.5 (microtask) → 3 (macrotask) → 4 (I/O)
```

## 1.2 I/O non-bloquante et callbacks

L'API originale de Node.js utilise les callbacks pour gérer l'asynchronisme. Un callback est une fonction passée en argument à une fonction asynchrone, appelée quand l'opération se termine. Le premier paramètre du callback est conventionnellement l'erreur (null si tout s'est bien passé) — c'est le

'Error-First callback pattern'.

#### ■■ Attention

Le 'callback hell' survient quand on imbrique trop de callbacks les uns dans les autres. Cette pyramide de code est difficile à lire et à maintenir. Les Promises et async/await ont été introduits pour résoudre ce problème — préférez-les dans tout nouveau code.

## 1.3 Async/Await — la syntaxe moderne

async/await est du sucre syntaxique sur les Promises. Une fonction marquée async retourne toujours une Promise. Le mot-clé await pause l'exécution de la fonction courante jusqu'à ce que la Promise soit résolue ou rejetée — sans bloquer l'event loop (l'exécution des autres callbacks continue normalement). C'est la façon idiomatique d'écrire du code asynchrone en Node.js moderne.

*Async/await et parallélisme avec Promise.all*

```
// Gestion d'erreurs avec async/await
async function getOrderWithItems(orderId: string, tenantId: string) {
  // await suspend CETTE fonction, pas l'event loop
  const order = await supabase
    .from('orders')
    .select('*', items:order_items(*, product:products(*))')
    .eq('id', orderId)
    .eq('tenant_id', tenantId)
    .single();

  if (order.error) {
    throw new NotFoundError('Order not found');
  }

  return order.data;
}

// Lancer plusieurs requêtes en parallèle
const [orders, products, stats] = await Promise.all([
  fetchOrders(tenantId),
  fetchProducts(tenantId),
  fetchStats(tenantId),
]);

// Les trois requêtes partent simultanément – 3x plus rapide que séquentiel
```

# Construire une API REST avec Express

## 2.1 Qu'est-ce que REST ?

REST (Representational State Transfer) est un style d'architecture pour les APIs web défini par Roy Fielding dans sa thèse de 2000. Une API RESTful utilise les méthodes HTTP standards, des URLs qui représentent des ressources (et non des actions), et renvoie des représentations JSON ou XML de ces ressources.

Les contraintes REST essentielles sont : sans état (chaque requête contient toutes les informations nécessaires — le serveur ne garde pas de session côté serveur), interface uniforme (HTTP + URLs cohérentes), système en couches (le client ne sait pas s'il parle directement au serveur ou à un proxy).

### Méthodes HTTP et leur sémantique REST

| Méthode HTTP | Sémantique             | Idempotent ? | Exemple                               |
|--------------|------------------------|--------------|---------------------------------------|
| GET          | Lire une ressource     | Oui          | GET /orders — liste des commandes     |
| POST         | Créer une ressource    | Non          | POST /orders — créer une commande     |
| PUT          | Remplacer entièrement  | Oui          | PUT /orders/123 — remplacer           |
| PATCH        | Modifier partiellement | Non          | PATCH /orders/123 — changer le statut |
| DELETE       | Supprimer              | Oui          | DELETE /products/456 — supprimer      |

## 2.2 Routing et méthodes HTTP

Express organise les routes par méthode HTTP et chemin URL. Le Router d'Express permet de regrouper les routes liées et de les monter sous un préfixe commun. Cela favorise la séparation des responsabilités : chaque fichier de routes gère une ressource distincte.

Structure d'un router Express avec TypeScript

```
// src/routes/orders.ts

import { Router } from 'express';
import { requireAuth } from '../middleware/auth';

const router = Router();

// Toutes les routes nécessitent une authentification
```

```

router.use(requireAuth);

// GET /orders?page=1&limit=20
router.get('/', async (req, res, next) => {
  try {
    const page = Number(req.query.page) || 1;
    const limit = Number(req.query.limit) || 20;
    const orders = await getOrders(req.tenant.id, { page, limit });
    res.json(orders);
  } catch (err) {
    next(err); // Déléguer à l'error handler
  }
});

// GET /orders/:id
router.get('/:id', async (req, res, next) => {
  try {
    const order = await getOrder(req.params.id, req.tenant.id);
    res.json(order);
  } catch (err) {
    next(err);
  }
});

export default router;

```

## 2.3 Paramètres, query strings, body

Express expose trois sources de données dans une requête : `req.params` pour les paramètres de route (`:id`), `req.query` pour les query strings (`?page=1`), et `req.body` pour le corps de la requête (JSON, formulaire). Le body n'est disponible qu'après l'application du middleware `express.json()`.

```

// URL : PATCH /orders/abc-123?notify=true
// Body : { 'status': 'delivered' }

router.patch('/:id', async (req, res, next) => {
  const orderId = req.params.id; // 'abc-123' – string
  const notify = req.query.notify; // 'true' – toujours string
  const { status } = req.body; // 'delivered'

  // Les query params sont TOUJOURS des strings

```



```
const shouldNotify = notify === 'true'; // conversion manuelle nécessaire

// ...
});

// Monter les routes dans app.ts
import express from 'express';
import ordersRouter from './routes/orders';

const app = express();
app.use(express.json()); // Parser le JSON body
app.use('/orders', ordersRouter); // Préfixe /orders
```

## Middleware — le cœur d'Express

---

### 3.1 La chaîne de middleware

---

Un middleware Express est une fonction qui reçoit req, res et next. Il peut lire et modifier req et res, terminer le cycle requête-réponse, ou appeler next() pour passer au middleware suivant. La chaîne de middleware s'exécute dans l'ordre de déclaration — ce détail est fondamental. Un middleware d'authentification doit être déclaré AVANT les routes qu'il protège.

```
// Anatomie d'un middleware

function myMiddleware(req, res, next) {
  // 1. Peut lire/modifier req
  console.log(`${req.method} ${req.path}`);

  // 2. Peut terminer la requête
  if (!req.headers.authorization) {
    return res.status(401).json({ error: 'Unauthorized' });
  }

  // 3. Ou passer au suivant
  next();
}

// Ordre d'exécution dans Express :
app.use(cors()); // 1er – CORS headers
app.use(express.json()); // 2e – parser le body
app.use(requestLogger); // 3e – logger
app.use('/api', router); // 4e – routes
app.use(errorHandler); // Dernier – gestion d'erreurs
```

### 3.2 Écrire ses propres middlewares

---

Les middlewares personnalisés sont l'outil le plus puissant d'Express. Le middleware d'authentification est l'exemple le plus courant : il vérifie le JWT, charge l'utilisateur depuis la base de données, et attache le tenant et l'utilisateur à req pour que les routes y aient accès.

*Middleware d'authentification avec Supabase*

```
// middleware/auth.ts
```

```

import { Request, Response, NextFunction } from 'express';
import { supabase } from '../lib/supabase';

// Augmenter le type Request pour inclure tenant et user
declare global {
  namespace Express {
    interface Request {
      tenant: { id: string; name: string };
      user: { id: string; role: string };
    }
  }
}

export async function requireAuth(req: Request, res: Response, next: NextFunction) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Missing token' });
  }

  const token = authHeader.slice(7);
  const { data: { user }, error } = await supabase.auth.getUser(token);

  if (error || !user) {
    return res.status(401).json({ error: 'Invalid token' });
  }

  // Charger tenant depuis la DB
  const { data: userRow } = await supabase
    .from('users')
    .select('tenant_id, role, tenant:tenants(id, name)')
    .eq('id', user.id)
    .single();

  req.user = { id: user.id, role: userRow.role };
  req.tenant = userRow.tenant;
  next();
}

```

### 3.3 Gestion des erreurs

---

Un error handler Express est un middleware spécial avec quatre paramètres (err, req, res, next). Il doit être déclaré EN DERNIER dans la chaîne de middleware. Quand une route ou un middleware appelle next(err) ou lève une exception dans un try/catch, Express passe directement à cet error handler en ignorant les middlewares intermédiaires.

*Système d'erreurs HTTP standardisé*

```
// lib/httpErrors.ts – classe d'erreurs personnalisées
export class HttpError extends Error {
  constructor(public status: number, message: string) {
    super(message);
  }
}

export class NotFoundError extends HttpError { constructor(m: string) { super(404, m); } }
export class UnauthorizedError extends HttpError { constructor(m: string) { super(401, m); } }
export class ConflictError extends HttpError { constructor(m: string) { super(409, m); } }

// middleware/errorHandler.ts
export function errorHandler(err, req, res, next) {
  if (err instanceof HttpError) {
    return res.status(err.status).json({ error: err.message });
  }

  // Erreur inconnue – ne pas exposer les détails en production
  console.error(err);
  res.status(500).json({ error: 'Internal server error' });
}
```

#### ■ À retenir

- ✓ L'event loop permet à Node.js de gérer des milliers de connexions avec un seul thread
- ✓ Toujours utiliser async/await — éviter les callbacks imbriqués
- ✓ REST : méthodes HTTP + URLs qui représentent des ressources
- ✓ La chaîne de middleware s'exécute dans l'ordre de déclaration — l'ordre compte
- ✓ Le middleware d'auth augmente req.tenant et req.user pour toutes les routes en aval
- ✓ Toujours appeler next(err) dans les try/catch — jamais laisser une erreur sans handler

#### — ■ Exercices

##### Exercice 1 — API Todo List complète

Construire une API REST complète pour une liste de tâches (Todo List).

- Créer les routes : GET /todos, POST /todos, PATCH /todos/:id, DELETE /todos/:id
- Ajouter un middleware d'authentification fictif (vérifier un header X-User-Id)
- Toutes les routes doivent retourner du JSON avec les bons codes de statut
- Implémenter un error handler global et des classes d'erreurs custom

### Exercice 2 — Middleware de validation

Créer un middleware générique de validation de body JSON.

- Le middleware prend un schéma Zod comme paramètre
- Il valide req.body contre le schéma et retourne 400 si invalide
- Il attache le body validé (et typé) à req.validatedBody
- L'appliquer sur les routes POST et PATCH

### Exercice 3 — Rate limiting par tenant

Implémenter un rate limiter qui limite chaque tenant à 100 requêtes/minute.

- Utiliser une Map en mémoire
- Le middleware vérifie le tenant\_id extrait du JWT
- Retourner 429 Too Many Requests avec le header Retry-After
- Tester avec plusieurs tenants simultanément

# Bibliographie & Ressources

---

## Documentation officielle

- [nodejs.org/docs](https://nodejs.org/docs) — documentation Node.js
- [expressjs.com/en/4x/api.html](https://expressjs.com/en/4x/api.html) — référence API Express 4
- [npmjs.com](https://npmjs.com) — registre des packages npm

## Livres

- Evan Hahn — Express in Action (Manning, 2016)
- Cassio de Sousa Antonio — Practical Node.js (Apress, 2018)

## Packages essentiels

- `express` — framework web minimaliste
- `zod` — validation de schémas TypeScript-first
- `jsonwebtoken` — création et vérification de JWT
- `express-rate-limit` — rate limiting simple
- `morgan` — logger HTTP pour Express
- `cors` — middleware CORS